TENANTPRO — SMART TENANT AND RENT MANAGEMENT SYSTEM

A project report submitted in partial fulfilment of the requirements of
Master of Computer Applications Post Graduate Course of
JAIN (Deemed-to-be University)

Submitted by

Anush Kulal M **24MCAR0133**
Akshita Bansal **24MCAR0131**
Chandan Mathur **24MCAR0230**

Under the guidance of

Dr. Kamalraj R

Department of Computer Science and IT
JAIN (Deemed-to-be University)
Bengaluru

Date -

CERTIFICATE BY THE UNIVERSITY

Certified that the project report titled “TenantPro — Smart Tenant and Rent Management System” is based on an original project study conducted by Anush Kulal M (Register No. 24MCAR0133), Akshita Bansal (Register No. 24MCAR0131), Chandan Mathur (Register No. 24MCAR0230) of the Master of Computer Applications programme, under the guidance of Dr. Kamalraj R. They have attended the required guidance sessions held. The project report has not formed a basis for the award of any Degree or Diploma of any university or any other institution.

PLACE: Bengaluru

DATE:

Dr. SAGAR GULATI
DIRECTOR
SCHOOL OF COMPUTER SCIENCE AND IT
JAIN (Deemed-to-be University)

CERTIFICATE BY THE GUIDE

Certified that the project report titled “TenantPro — Smart Tenant and Rent Management System” is based on an original project study conducted by Anush Kulal M (Register No. 24MCAR0133), Akshita Bansal (Register No. 24MCAR0131), Chandan Mathur (Register No. 24MCAR0230) under my guidance. This project has not formed a basis for the award of any Degree or Diploma of any University or any other Institution.

PLACE: Bengaluru

DATE:

Signature of the Guide

Dr. Kamalraj R

Department of Computer Science and IT

DECLARATION BY THE STUDENTS

This is to declare that the project titled “TenantPro — Smart Tenant and Rent Management System” is carried out independently by us under the guidance of Dr. Kamalraj R, Department of Computer Science and IT, JAIN (Deemed-to-be University), Bengaluru. This work is an original one and has not been submitted earlier to any University or any other Institution for the fulfilment of the requirement of a course of study or any other credential.

PLACE: Bengaluru

DATE:

NAME: Anush Kulal M
USN: 24MCAR0133

NAME: Akshita Bansal
USN: 24MCAR0131

NAME: Chandan Mathur
USN: 24MCAR0230

ACKNOWLEDGEMENT

We are deeply indebted to Dr. CHENRAJ ROYCHAND, President, JAIN (Deemed-to-be University) Trust, Bengaluru, for having admitted us to undergo the Master of Computer Applications Post Graduate course during the academic term 2025 - 2026.

We are pleased to thank Dr. Sagar Gulati, Director, School of Computer Science and IT, JAIN (Deemed-to-be University), Bengaluru, for the facilities and the academic environment extended to us during the course of this work.

We take this opportunity to express our gratitude to Dr. Ananta Charan Ojha, Deputy Director, Dr. K Suneetha, Head of the Department of Computer Science and IT, and Dr. Murugan R, Programme Head of the MCA Programme, for their valuable guidance and support for the successful completion of the project work.

We express our profound thanks to our project coordinator Dr. I Brem Navas, Professor, Department of Computer Science and IT, and above all to our guide Dr. Kamalraj R, for guidance at the decisive points of this work and for being available whenever it mattered.

We are very thankful to our families and friends for their constant and unconditional encouragement and support.

NAME: Anush Kulal M
USN: 24MCAR0133

NAME: Akshita Bansal
USN: 24MCAR0131

NAME: Chandan Mathur
USN: 24MCAR0230

TABLE OF CONTENTS

Chapter	Title	Page No.
	Abstract	1
1	Introduction	3
	1.1 Overview	3
	1.2 Problem Background	3
	1.3 Motivation	4
	1.4 Objectives of the Project	4
	1.5 Scope of the Project	5
	1.6 Organization of the Report	6
2	Literature Study	7
	2.1 Introduction	7
	2.2 Review of Traditional Manual Systems	7
	2.3 Review of Existing Digital Rental Applications	7
	2.4 Payment Models on Two-Sided Platforms	8
	2.5 Review of Token-Based Session Management	9
	2.6 Gap Analysis	10
3	System Analysis	11
	3.1 Existing System	11
	3.2 Limitations of the Existing System	11
	3.3 Proposed System	12
	3.4 Advantages of the Proposed System	12
	3.5 Feasibility Study	13
4	System Design and Architecture	15
	4.1 System Architecture	15
	4.2 Use Case Diagram	15
	4.3 Activity Diagram for Paying Rent	16
	4.4 Data Flow Diagrams	18
	4.5 Client Architecture and the View-Model	19
	4.6 API and Authorisation Layer	20
	4.7 Payment and Settlement Design	20
	4.8 Session and Authentication Design	21
	4.9 Database Design	22
5	System Requirements	24
	5.1 Hardware Requirements	24
	5.2 Software Requirements	24
	5.3 Programming Platform Features	25

Chapter	Title	Page No.
	5.4 Functional Requirements	25
	5.5 Non-Functional Requirements	26
6	Implementation	27
	6.1 Authentication and Session Module	27
	6.2 Property, Room and Tenant Management Module	27
	6.3 Rent Cycle and Due Date Module	28
	6.4 Payment and Confirmation Module	28
	6.5 Payment Gateway Connection Module	28
	6.6 Notification and Reminder Module	29
	6.7 Maintenance Request Module	29
	6.8 Tenant Behaviour Score Module	29
	6.9 Tenant Portal Module	30
	6.10 Testing	30
7	Results and Screenshots	34
	7.1 Landlord Overview	34
	7.2 Tenant Roster	35
	7.3 Rooms and Properties	36
	7.4 Ledger	37
	7.5 Connecting a Payment Gateway	38
	7.6 Tenant Home	39
	7.7 Paying Rent	40
	7.8 Tenant Profile and Payment History	41
	7.9 Maintenance	42
8	Conclusion and Future Enhancement	44
	8.1 Conclusion	44
	8.2 Limitations of the Present Work	45
	8.3 Future Enhancement	45
	References	47

LIST OF TABLES

Table No.	Title	Page No.
2.1	Comparison of Reviewed Categories of Existing Systems	8
2.2	Structural Options for Moving Money on a Platform	9
3.1	Existing System Compared with the Proposed System	12
3.2	Feasibility Analysis Summary	14
4.1	Evolution of the Session Design	21
4.2	Principal Tables and Their Role	22
5.1	Hardware Requirements	24
5.2	Software Requirements	24
6.1	Test Suites and Assertion Counts	30
6.2	Defects Found by the End-to-End Runs	32

LIST OF FIGURES

Figure No.	Title	Page No.
4.1	System architecture: client, API, data, supporting services, and the money path outside all of them	15
4.2	Use case diagram: the landlord, the tenant, and the two external systems	16
4.3	Activity diagram: the two payment routes converging on a single settlement step	17
4.4	Data flow diagram, level 0: the context diagram	18
4.5	Data flow diagram, level 1: processes, data stores and the external systems	19
4.6	Entity relationship diagram of the core schema	23
7.1	Landlord overview: collection against expectation, overdue total, occupancy, six-month revenue and the open-ticket queue	35
7.2	Tenant roster with counted filters, due-state chips and behaviour scores	36
7.3	Room list showing occupancy, rent and tenant assignment per unit	37
7.4	Ledger of recorded payments with method, date and tenant	38
7.5	The online payments sheet: what it costs, what the gateway will ask for, and the key fields	39
7.6	Tenant home: rent due with countdown, deposit held, tenure, open requests and the landlord's contact	40
7.7	The payment screen with a gateway connected: the online route above the divider and the UPI route beneath it	41
7.8	Tenant profile: tenure, rent share, deposit, the behaviour score with its factors, and the payment timeline	42
7.9	Tenant help screen listing maintenance requests with their status	43

ABSTRACT

Rental housing is one of the largest parts of the urban economy in India and one of the least organised. Paying guest establishments, private hostels and small apartment blocks house a very large share of the students and working people in every city, and almost all of them are still run on paper. Names go into a register at the front desk. Rent arrives by UPI transfer or in cash. The record of a payment is a screenshot sitting on somebody's phone. A tenant who has paid holds no receipt their landlord has ever acknowledged, and a landlord with six rooms usually cannot say with confidence who is behind this month.

TenantPro was built to replace that arrangement. It has two sides: a landlord application and a tenant portal, both served from one React Native codebase running on Expo SDK 54, talking to a Node and Express API over MySQL. The system is deployed and in use. At the commit documented here it comes to 38,536 lines across 124 source files, exposes 87 API endpoints over 22 database tables, and presents 28 screens.

One decision shapes the rest of the design. Money never passes through the platform. Rent goes straight from the tenant to the landlord, either by UPI into the landlord's own address or through a payment gateway connected to the landlord's own merchant account. That keeps the operator outside the Reserve Bank of India's payment-aggregator perimeter, which carries an authorisation requirement and a net-worth floor in the tens of crores, and it means the platform bills nobody and holds nobody's funds. The catch is that an application which does not touch the money also cannot see it. So every payment starts life as a claim and stays one until a second party vouches for it: the landlord tapping Confirm, or the gateway answering a direct query from the server. Both routes end in the same settlement function, so they cannot disagree about when a month has cleared.

The build carries 457 assertions across eight test suites. Of those, 168 run end to end against a live server and a real MySQL instance, and 71 drive the compiled application inside a browser engine. Those runs found five defects the unit tests had missed, one of them a cross-account authorisation hole. All five are fixed. This report sets out the architecture, the features as they actually stand, the payment and session designs, how the system was tested, and what has been identified but not built.

Keywords: Rent Management, Property Management System, Tenant Portal, Direct Payment Model, Payment Aggregator Regulation, UPI, Razorpay Payment Links, Refresh Token Rotation, React Native, Expo, Node.js, MySQL, End-to-End Testing.

CHAPTER 1 — INTRODUCTION

1.1 Overview

Rented accommodation carries a large part of the urban housing load in India. Paying guest establishments, private hostels and small apartment blocks of anywhere between two and twenty rooms are where most students and most young working people live. Nearly all of these properties belong to individuals rather than to companies, and nearly all of them are managed without software of any kind.

A typical property runs out of a register. Names are written in when somebody moves in and struck out when they leave. Rent comes by UPI or in cash, and the only trace of it is a transaction screenshot the tenant forwards over WhatsApp. Deposits are remembered rather than written down. Maintenance complaints arrive as phone calls and are either dealt with or forgotten, with nothing recorded either way. None of this is a problem at one property with four rooms. It becomes one at two properties with twelve.

TenantPro replaces that patchwork with a single record both sides can see. The landlord gets a roster of properties, rooms and tenants, with due dates worked out from each tenancy's own anniversary. The tenant sees the same records from the other direction: what they owe, when it falls due, what they have paid, and which of it the landlord has acknowledged. The client is a React Native application shipped as an Android build. The server is a Node and Express API over MySQL, deployed and running.

1.2 Problem Background

Anybody who has run a rented property for a year or two will recognise the problems with doing it by hand. Rent amounts get recorded inconsistently, or not at all. A payment made on the first and a payment made on the nineteenth look identical in the register once both have been ticked. When a tenant insists they have paid and the owner cannot find the entry, neither side has anything useful: the tenant has a screenshot proving a transfer happened, which says nothing about which month it was meant for.

Deposits are the worst version of the same thing. A deposit is collected at the start of a tenancy that might run two years and is settled at the end of it. In between, the only record is a figure in the owner's head or a line in a notebook that has since been thrown out.

Arguments at move-out are common, and the tenant usually loses them, because the tenant has no record at all.

On the owner's side, what limits growth is the lack of a consolidated view. One property can be held in your head. Three cannot, and the first thing to go is a reliable answer to the simplest question in the business: who has not paid. Reminders then go either to everybody, which irritates the people who have already paid, or to nobody, which is why collection slips.

1.3 Motivation

The idea came from watching both sides fail at the same task for opposite reasons. Owners described losing the first week of every month to chasing payments they could not confirm. Tenants described paying on time and then being asked again a week later with no way to prove the first payment had landed.

The surrounding conditions have also changed a great deal in the last five years. UPI has made digital payment ordinary rather than exceptional, so the movement of money is already solved and does not need reinventing. Cloud hosting is cheap. Practically every tenant carries a smartphone. What is missing is not a way to pay but a shared record of what the payment was for.

Looking at the applications already in this space showed a consistent split. The simpler ones are digital notebooks with no tenant-facing side, which leaves the tenant exactly where the paper register left them. The more ambitious ones fix the record problem by routing the rent through their own account and settling it on to the landlord afterwards. That solves visibility by creating a regulatory problem, and it also means the owner's rent spends time in somebody else's account. Neither approach suited the situation this project set out to serve.

1.4 Objectives of the Project

The project was shaped around a set of objectives.

1. To replace manual record keeping with a structured digital record of properties, rooms, tenants, tenancies, rent, deposits and payments, held in a relational database instead of a register.

2. To give the tenant a real view of that record, so that the person paying the rent can see what is owed, what has been paid and what the landlord has formally acknowledged.
3. To work out each tenancy's due date from its own start date rather than from a shared calendar month, so that somebody who moved in on the nineteenth is due on the nineteenth.
4. To make digital payment the ordinary route, both by UPI straight into the landlord's account and through a gateway connected to the landlord's own merchant account.
5. To keep the platform out of the flow of funds completely, so that it never holds or transmits a tenant's money and does not fall inside the regulatory perimeter that applies to payment aggregators.
6. To make sure no payment is ever treated as settled on the payer's own say-so, by requiring confirmation from the landlord or the gateway before a rent cycle moves forward.
7. To send rent reminders automatically on the channels tenants actually use, and to give maintenance complaints a shared thread so that they are recorded rather than remembered.
8. To build a session design in which a session can actually be ended, so that a stolen phone, a tenant who has moved out or a password change has some real effect.
9. To test the result by driving every endpoint against a real database and the compiled application against a real browser engine, instead of relying on unit tests over mocks.

1.5 Scope of the Project

The scope covers the operational life of an independently owned rental property. That means registering a landlord account, creating properties and rooms, bringing tenants on board either by direct entry or by an invite code they redeem, recording rent and deposit terms, generating due dates, sending reminders, accepting and confirming payments, keeping a searchable ledger, running maintenance threads, holding identity documents, computing a payment-behaviour score both parties can see, recording notice periods and leave dates, and giving the tenant a portal where all of it is visible from their side.

Some things were deliberately left out. The system does not hold or move money and is not an escrow. It does not draft, store or register rental agreements, and makes no claim on the legal side of a tenancy. It is not a listing marketplace and does no brokerage. There

are no accounting exports and no tax filing. Expense and vendor management are outside the present scope. And the client is an Android application; no iOS build has been made.

1.6 Organization of the Report

Seven chapters follow. Chapter 2 reviews the manual practices and the digital applications already in this space, together with the regulatory and security material that constrains any design here, and ends with a gap analysis. Chapter 3 analyses the existing arrangement, lists its limitations, describes what is proposed instead, and reports a feasibility study run along four axes. Chapter 4 covers the architecture, the use case, activity and data flow models, and the design of the payment, session and database layers. Chapter 5 states the hardware, software, functional and non-functional requirements. Chapter 6 goes through the implementation module by module and reports the testing and its results. Chapter 7 shows the finished interface with screenshots from the running application. Chapter 8 concludes, states the limitations honestly, and sets out what has been identified for later.

CHAPTER 2 — LITERATURE STUDY

2.1 Introduction

A project like this starts by looking at what already exists. Three bodies of material mattered here: the informal practices that currently run the independent rental sector, the digital applications built to replace them, and the regulatory and security literature that constrains how anybody may build in this space. The third turned out to matter more than the second, because the central architectural decision in this project follows directly from it.

2.2 Review of Traditional Manual Systems

There is a good deal of management literature on why small businesses stay on manual systems even when digital alternatives are cheap and available. The findings repeat. What the owner weighs is not the price of the software but the time and risk of moving to it. The owner is usually also the operator and has no spare capacity for a migration. And manual systems fail quietly, so nobody ever measures what they cost.

Work on small hospitality and rental businesses makes a related point: manual records understate actual losses. A missed payment, a partial payment nobody followed up, a deposit returned in full when deductions were due — none of these show up as losses anywhere, because there is no record to compare them against. The owner's sense of the business is therefore reliably rosier than its accounts would be, which weakens the case for changing anything.

The useful conclusion for this project was not that owners are being irrational. It was that any replacement has to be usable on day one without a migration project, and has to show its value in a way the owner can see. Several decisions later in the design follow from that, in particular the choice to let a landlord run the whole system with no payment gateway connected and still get the benefit of the record.

2.3 Review of Existing Digital Rental Applications

A fair number of applications now target the Indian rental market. They range from simple mobile apps built by individual developers, through mid-tier rent-collection products, up to full property management suites aimed at organised co-living operators. Reviewing them showed a split that runs right through the category.

The simpler applications are digital registers. They give the owner somewhere to type what they would otherwise have written, which is a genuine improvement, but there is no tenant-facing side at all. The tenant ends up where they started, with no acknowledged receipt and no view of their own history. Several also model a property as a flat list of tenants, with no notion of a property holding rooms holding occupants, which makes them unusable past one small building.

The more capable products do have a tenant side, and they solve the confirmation problem convincingly, because they collect the rent themselves. The tenant pays the platform, the platform marks the payment received and then settles the money on to the landlord. From a software point of view this is clean. It also places the platform squarely inside the regulatory perimeter discussed in the next section, and it means the landlord's rent passes through an account that is not theirs on the way to them. Several of the owners we spoke to during this project objected to that without being prompted.

Table 2.1 — Comparison of Reviewed Categories of Existing Systems

Category	Tenant-facing view	Property hierarchy	Handles money	Suitability for independent owner
Paper register	None	Implicit, in the owner's head	No	Workable at one small property only
Simple digital notebook apps	None or read-only list	Flat tenant list	No	Breaks down beyond one property
Rent collection platforms	Yes	Partial	Yes, collects and settles	Regulated activity; owner loses direct settlement
Organised co-living suites	Yes	Full	Yes	Priced and scoped for operators, not individuals
TenantPro	Full parallel portal	Property to room to tenant	No, payment is direct	Designed for this segment

2.4 Payment Models on Two-Sided Platforms

Any platform sitting between a payer and a payee has three structural options, and choosing between them is a regulatory decision before it is a technical one.

Under the aggregator model the platform receives the customer's money into its own account and then pays the merchant. The Reserve Bank of India's directions on payment aggregators make this a regulated activity. An aggregator cannot keep customer funds in its ordinary account. The money has to sit in a segregated escrow with a scheduled commercial bank, it earns the platform no interest, and it has to be settled out within

prescribed timelines. Getting the authorisation at all means clearing a net-worth floor in the tens of crores. Ride-hailing platforms are the familiar example.

Under split settlement the platform does not hold the money itself. Instead it instructs a licensed gateway to split an incoming payment between a platform account and a linked sub-merchant account. The gateway holds the licence and runs the know-your-customer checks on the sub-merchants. Razorpay Route is the common implementation in India. This takes the onboarding burden off the platform and is attractive at scale, but funds still pass through a platform-controlled account, and the 2025 directions restrict how such arrangements may be put together.

Under the direct model the platform never touches the money. The payer pays the payee. All the platform does is present the right amount, the right destination and a reference, and record what happened afterwards. Namma Yatri is the best-documented Indian precedent: a ride-hailing service where the fare goes straight from rider to driver and the platform takes no commission, sustaining itself some other way. TenantPro sits here.

Table 2.2 — Structural Options for Moving Money on a Platform

Model	Who holds the funds	Authorisation required	Example
Aggregator	The platform, in a ring-fenced escrow at a scheduled commercial bank	RBI payment aggregator licence	Uber, Ola
Split settlement	The gateway, settling to linked sub-merchant accounts	Held by the gateway, not the platform	Razorpay Route
Direct	Nobody; the payer pays the payee	None	Namma Yatri, TenantPro

2.5 Review of Token-Based Session Management

The security literature on sessions for mobile clients turns on one trade-off: stateless tokens against stateful ones. A signed access token of the JSON Web Token family is checked by its signature alone, with no database lookup. That is what makes it fast and what lets it scale. The price is that nothing can take it back. A stolen device, a tenancy that has ended, a password somebody just changed — none of these affect a token already in circulation. It runs until it expires.

Current practice, set out in the OAuth 2.0 security guidance, pairs a short-lived access token with a long-lived refresh token held on the server, so that revocation has

somewhere to act. Two further properties are recommended. Rotation: presenting a refresh token spends it and returns a replacement, so a long-lived secret is never sent across the network over and over. And reuse detection: since rotation means the honest client is always holding the newest token, a token that has already been spent turning up again implies that a copy exists somewhere it should not, and the safe answer is to end every session on that account.

The literature is equally clear that the client half is where this usually goes wrong. A client that signs the user out on any authentication-looking error will sign them out on a dropped connection. A client that fires several requests at once on a cold start, all failing on the same expired token, will issue several refreshes, and all but the first will replay a spent token — which a correctly implemented server reads as theft. Both failure modes were designed against here and both are covered by tests.

2.6 Gap Analysis

Putting the three strands together, the gap is reasonably clear. Manual systems still dominate the independent rental sector despite being demonstrably lossy, because the cost of switching is real and the losses are invisible. The simpler digital applications cut the owner's effort but leave the tenant where they were, which means the dispute the system most needs to prevent is the one thing it does nothing about. The more capable platforms do give the tenant a real view, but they get there by taking custody of the rent, which is regulated and is in any case unattractive to the owners this project is for.

What nobody is offering is a system that gives both sides a shared, authoritative record of a tenancy while leaving the money exactly where it already flows, from the tenant's account into the landlord's. That is the gap TenantPro fills, and the confirmation model described in Chapter 4 is how it stays accurate about payments it is deliberately unable to watch.

CHAPTER 3 — SYSTEM ANALYSIS

3.1 Existing System

The arrangement we found across the properties studied for this project is a physical register at the property, a notebook or spreadsheet with the owner, a WhatsApp thread per tenant, and the owner's memory holding the rest together. New tenants go into the register with a phone number and a room. Rent terms are agreed verbally, or in a short written agreement that is then filed and never looked at again. Payment is by UPI or cash. A UPI payment produces a confirmation in the tenant's own banking app, which they screenshot and send across; a cash payment produces nothing at all.

Reminders go out by hand, usually in a burst at the start of the month, and usually to everybody, because working out who has already paid takes longer than just sending the message. Maintenance complaints come in by phone or WhatsApp and are tracked, if at all, in the same thread as everything else. Deposits are recorded once at the beginning and recalled at the end. Notice periods are settled in conversation.

At best a diligent owner keeps all this running with the occasional error. More often entries are missed, payments get chased twice, a deposit figure is disputed at move-out, and a complaint made three weeks ago has been forgotten by both parties.

3.2 Limitations of the Existing System

Accuracy goes first. Every figure is copied by hand at least once and some are never written down at all. Timeliness goes next: the owner's picture of who has paid lags reality by however long it takes to reconcile a pile of screenshots against the register. Evidence is one-sided, since the tenant holds proof that a transfer happened but no proof of what it settled, while the owner holds a register entry the tenant has never seen.

Scalability does not degrade gradually. It fails at a threshold, somewhere around the second or third property, which is exactly where an owner would otherwise expand. There is no accountability, because nothing records who entered what or when. And the tenant, who is after all the paying customer of the business, gets no interface whatsoever. That last point is the root of most of the disputes the arrangement produces.

3.3 Proposed System

The proposed system swaps the register and the notebook for a relational record, and puts an application over it for each party. Properties hold rooms, rooms hold tenancies, and each tenancy carries its own rent, deposit, start date and split across occupants. Due dates come from the tenancy's own start date, so a tenant who moved in on the nineteenth is due on the nineteenth and not on the first.

Payments are recorded against the due date they settle, captured at the moment the payment is declared. A payment enters as a claim and becomes real only when a second party vouches for it. Where the landlord has connected a gateway, the second party is the gateway, asked directly by the server using that landlord's own credentials. Where they have not, it is the landlord, who confirms or rejects with a reason the tenant can read. Either way the same settlement function moves the tenancy on.

The tenant portal is a parallel application, not a stripped-down copy of the landlord's. It answers a different question, and it is reached through a separate account type with its own table and its own token role. Reminders go out automatically across email, push, SMS and WhatsApp. Maintenance requests become threads the two parties share, with status changes recorded in sequence alongside what was said. A payment-behaviour score is computed once and shown identically to both sides, so neither is arguing with a number the other cannot see.

3.4 Advantages of the Proposed System

Figures are computed rather than copied, so the arithmetic stops being a source of error. The landlord's view of collection is built from the same rows the tenant's view is built from, which removes the lag. Confirmation becomes an explicit recorded act instead of an assumption, so both sides hold the same evidence. A second and third property cost nothing but data entry. And every payment, confirmation, rejection and status change is a row with a timestamp on it, which is as much accountability as a system of this size needs.

Table 3.1 — Existing System Compared with the Proposed System

Aspect	Existing system	Proposed system
Record of tenancy	Register entry and memory	Relational record, visible to both parties
Due date	Shared calendar month, applied loosely	Derived from each tenancy's own start date

Aspect	Existing system	Proposed system
Proof of payment	Tenant's screenshot of a transfer	Confirmed payment row carrying the due date it settled
Who confirms	Nobody; assumed on sight of a screenshot	Landlord, or the gateway asked server-to-server
Reminders	Sent by hand, usually to everybody	Automated daily job, addressed to those actually due
Deposit record	Remembered	Stored, and shown on the tenant's own screen
Maintenance	Phone call, tracked informally	Shared thread with recorded status transitions
Tenant's view	None	Full portal: dues, history, receipts, score, documents
Scalability	Fails at two or three properties	Unaffected by portfolio size

3.5 Feasibility Study

Feasibility was examined along four axes. The fourth is not one of the standard three, but it decided more about this project than the others did.

3.5.1 Technical Feasibility

Everything the system needs already exists and is well documented. React Native with Expo gives a single codebase targeting Android, with over-the-air updates for JavaScript changes. Node and Express over MySQL is an unremarkable server stack. Payment links, transactional email, push notifications and messaging are all reachable as ordinary HTTPS APIs. Nothing here had to be built from first principles. One technical constraint did shape the design and is picked up again in Chapter 4: native modules cannot be delivered over the air, so any feature needing a native software development kit would hold back the release cadence of everything queued behind it.

3.5.2 Economic Feasibility

Running costs are small. Hosting for the API and the database, transactional email, push delivery and media storage all sit within free or low tiers at the scale of a few properties, and they scale gracefully after that. Because the platform processes no payments, it carries no payment-processing cost of its own. Where a landlord connects a gateway, the gateway's fee is charged by the gateway to that landlord against their own merchant account; the platform is not a party to it and bills nothing. The one recurring cost identified on the roadmap is a verified sending domain for transactional mail, at roughly a thousand rupees a year.

3.5.3 Operational Feasibility

This is the axis we examined most carefully, because the people who will use the system are not necessarily comfortable with software. Two design consequences came out of it. The first is that the system has to be fully useful with no gateway connected, since gateway onboarding runs through a third party's verification process and takes days; a landlord who cannot use the product until that finishes is not coming back. The second is that any screen asking for a commitment has to state its cost and its prerequisites before the user starts, because a landlord who expects five minutes and runs into a week concludes the application is broken rather than that the gateway is slow.

3.5.4 Regulatory Feasibility

The regulatory axis settled the architecture. Collecting rent centrally and paying it out is payment aggregation under the Reserve Bank of India's directions, with the escrow obligations and the net-worth floor described in Chapter 2. A student project cannot meet those conditions. Neither can the individual landlords this system serves. So it was designed so that the question never comes up: the money moves straight from the tenant to the landlord, by UPI or through the landlord's own merchant account, and the platform neither receives it nor tells anybody to disburse it.

Table 3.2 — Feasibility Analysis Summary

Axis	Finding	Consequence for the design
Technical	All components mature and documented	Native modules avoided so that updates ship over the air
Economic	Low fixed cost; no payment-processing cost borne by the platform	Platform bills nobody; gateway fees are the landlord's own
Operational	Users are not necessarily technical; onboarding friction is fatal	System fully usable with no gateway connected
Regulatory	Central collection would be payment aggregation	Direct tenant-to-landlord payment; platform outside the flow of funds

CHAPTER 4 — SYSTEM DESIGN AND ARCHITECTURE

4.1 System Architecture

TenantPro uses a three-tier design. The presentation tier is a React Native client that serves the landlord application and the tenant portal from one codebase. The application tier is a Node and Express API organised as routers, each with its role guard mounted once at the top. The data tier is MySQL, with 22 tables covering accounts, properties, rooms, tenancies, payments, maintenance threads, documents, notifications and sessions.

Around the three tiers sit the supporting services: a transactional mail provider, push delivery, media storage for identity documents and maintenance photographs, and the payment gateway. The last one is drawn differently in the figure below, for a reason. Every piece of information about a tenancy goes through the server. No part of the payment does. The dashed band in the figure is the money path, and it touches the tenant and the landlord and nothing in between.

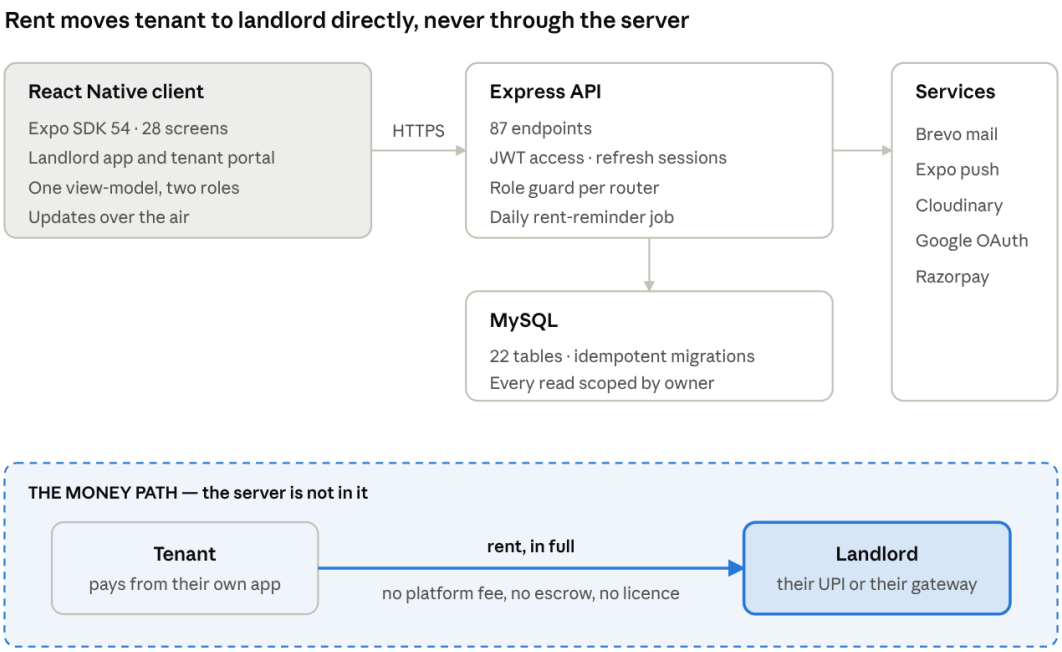


Fig 4.1 — System architecture: client, API, data, supporting services, and the money path outside all of them

4.2 Use Case Diagram

The use case diagram identifies the two primary actors and the two external systems the application talks to. The landlord and the tenant both sign in and hold a session, and from

there their paths separate almost completely: the landlord describes and administers the portfolio, while the tenant looks at their own position in it and acts on it.

Two things in the diagram are worth pointing out. The payment gateway appears as an external actor rather than as part of the system, which is the modelling consequence of the decision described in Chapter 2: the system asks the gateway questions and never handles the money itself. And the messaging services are external too, because the system composes a reminder but does not deliver it.

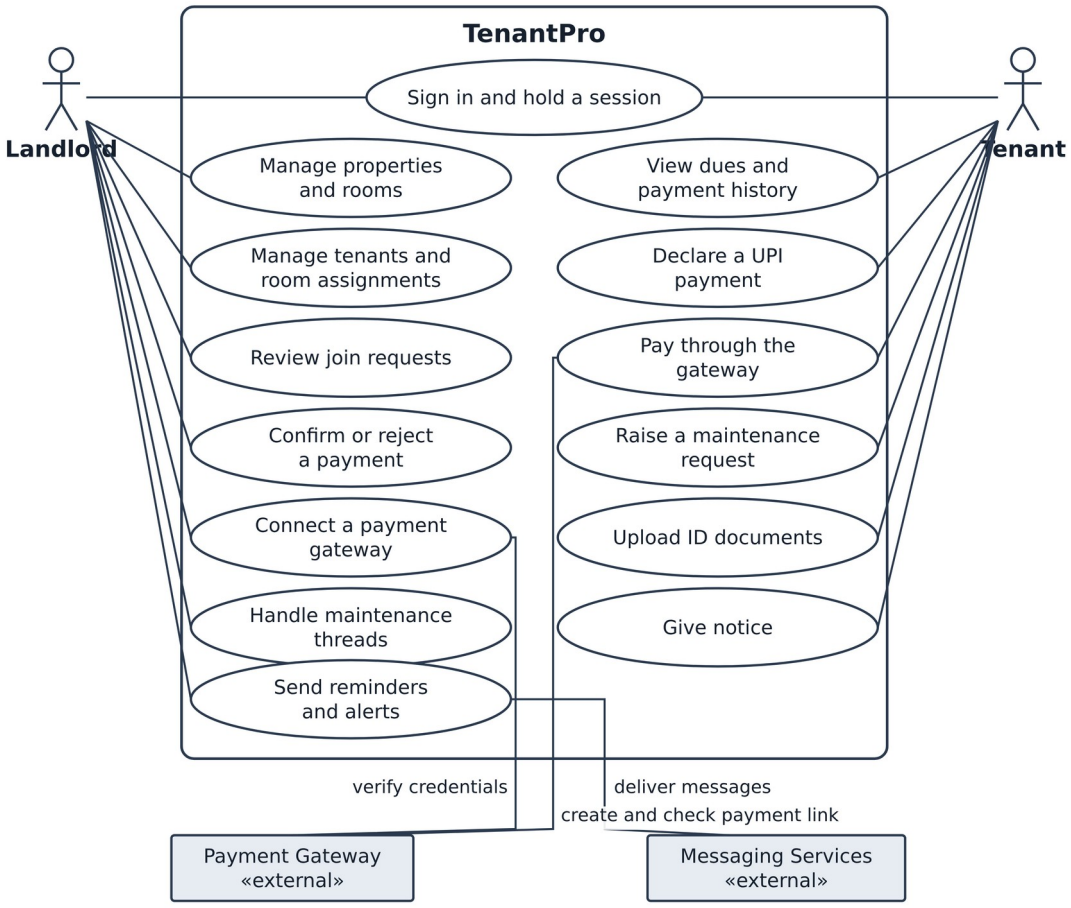


Fig 4.2 — Use case diagram: the landlord, the tenant, and the two external systems

4.3 Activity Diagram for Paying Rent

The activity diagram below traces a rent payment from the moment the tenant opens the payment screen to the moment both sides see it settled. The single decision at the top, whether the landlord has connected a gateway, is what sends the flow down one of two branches.

On the UPI branch the money moves outside the system entirely, so the tenant declares the payment afterwards and it sits as a claim until the landlord confirms or rejects it. On the gateway branch the server can ask the gateway directly, so the confirmation comes from the gateway rather than from a person. The important feature of the diagram is that the two branches converge: whichever route was taken, settlement happens in exactly one place, inside one transaction that both changes the status and advances the due date.

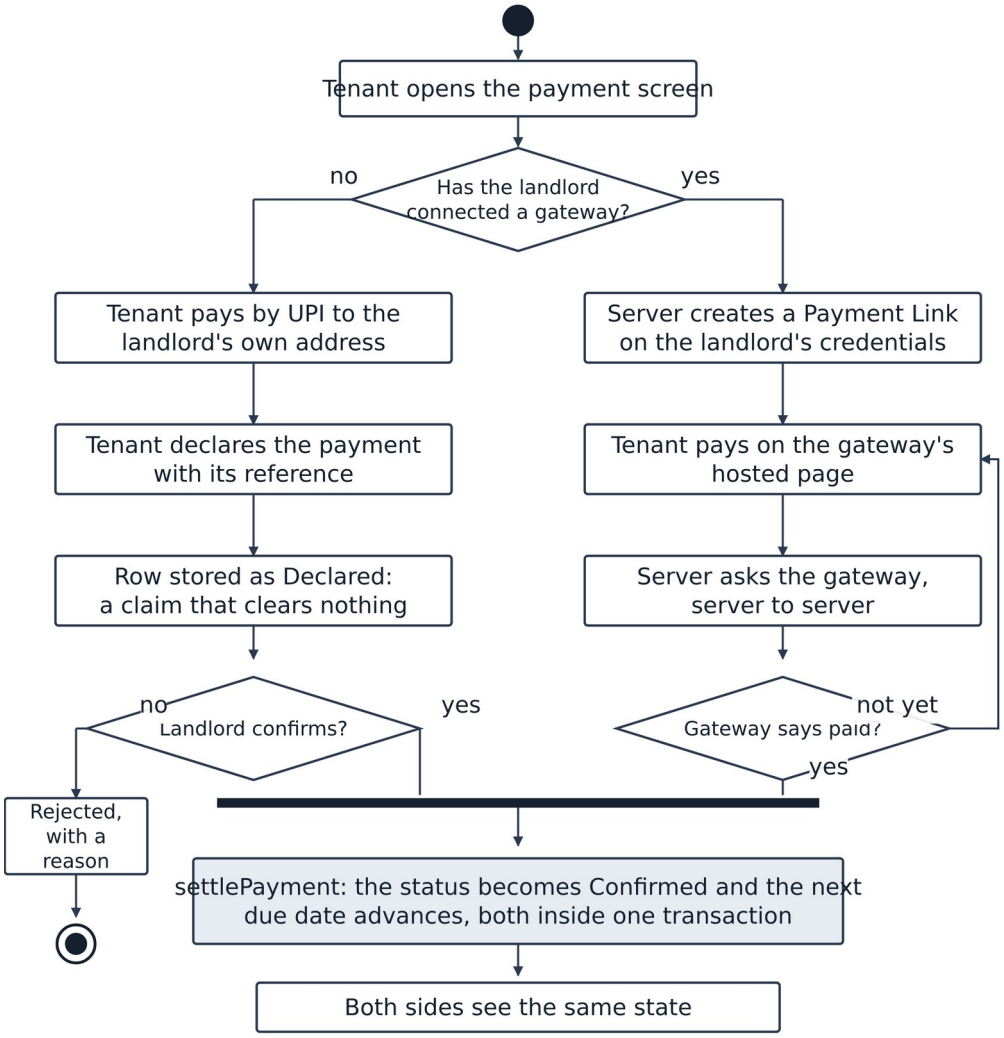
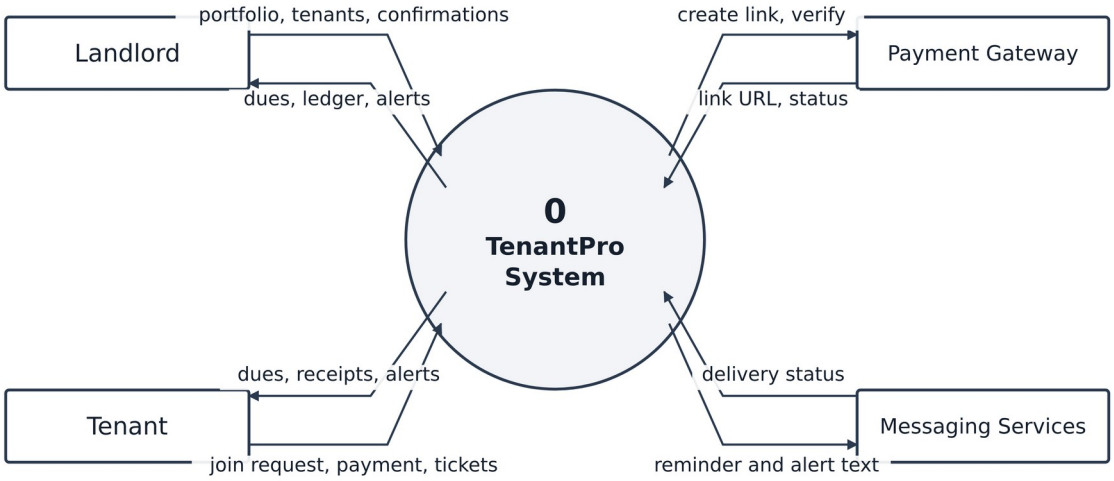


Fig 4.3 — Activity diagram: the two payment routes converging on a single settlement step

4.4 Data Flow Diagrams

The context diagram fixes the boundary of the system. Four external entities surround it: the landlord and the tenant, who supply and consume information, and the payment gateway and the messaging services, which the system calls out to. The flows are all information. No flow on this diagram carries money, which is the point the architecture turns on.

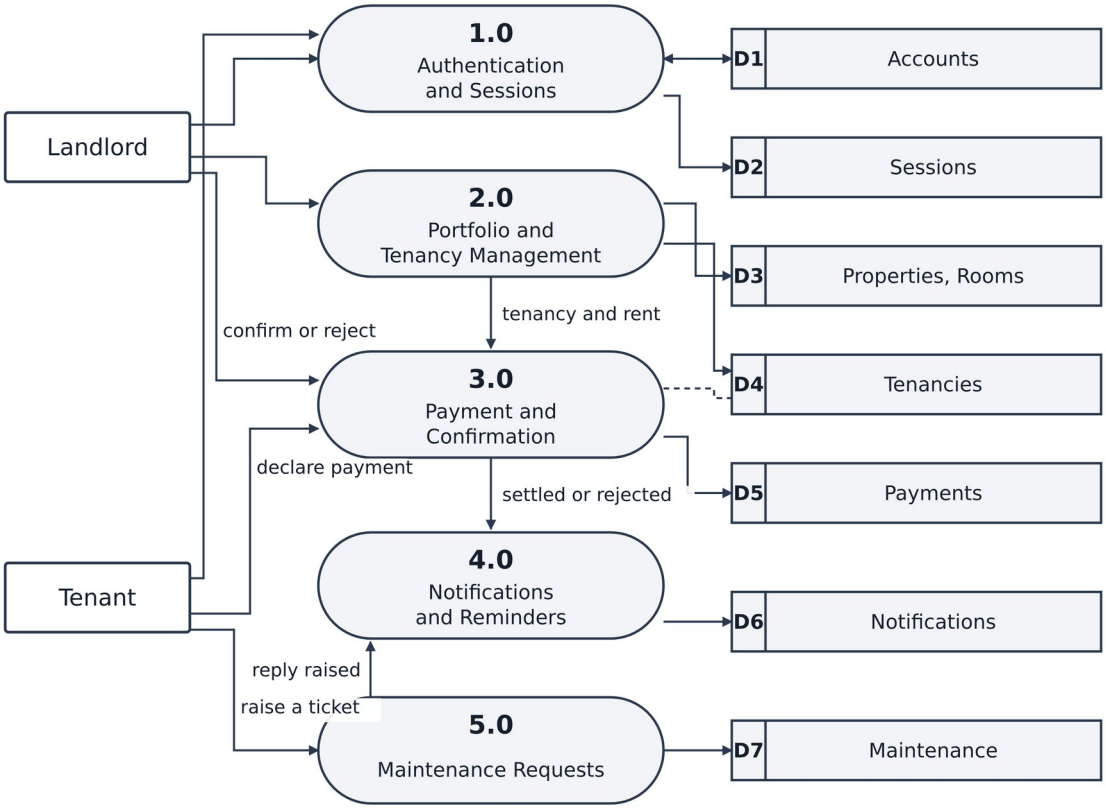


No flow on this diagram carries money: the gateway settles directly to the landlord.

Fig 4.4 — Data flow diagram, level 0: the context diagram

The level 1 diagram breaks the single process of the context diagram into five: authentication and sessions, portfolio and tenancy management, payment and confirmation, notifications and reminders, and maintenance requests. Seven data stores sit beside them.

The diagram also records a constraint that matters for correctness. Several processes read the tenancy store, but only process 3.0 writes a due date to it, and it does so only as part of the settlement transaction. Allowing any other process to move a due date would make it possible for a tenancy to advance without a payment behind it.



Dashed: process 3.0 reads tenancy rows and is the only process that advances a due date.

Fig 4.5 — Data flow diagram, level 1: processes, data stores and the external systems

4.5 Client Architecture and the View-Model

One React Native codebase serves both roles. Rather than spreading presentation logic across screens, the application runs the current state and the server payload through a single pure function that produces a view-model. A screen then holds layout and very little else.

Two things follow. Because the function is pure and needs no device, the behaviour of the interface can be asserted directly, and one of the suites in Chapter 6 does exactly that. And two screens showing the same fact cannot drift apart, since both read the same derived value instead of each working it out for itself.

The delivery mechanism imposed a constraint that shaped what got built. Updates reach an installed client through the Expo updates mechanism, which can swap out JavaScript but not native modules. Anything depending on a native software development kit can only go out in a full store release, and would hold back every other change waiting behind

it. Features of that kind were turned down wherever a design existed that ships like everything else. The payment integration is the clearest case and is covered in section 4.7.

4.6 API and Authorisation Layer

The API is organised as routers, each with its role guard mounted once at the top instead of repeated in every handler. Landlord and tenant accounts live in separate tables with their own auto-increments, so knowing who somebody is does not by itself settle what they may do. A tenant token on a landlord route is turned away by the guard, and tenant handlers scope every query through the link from the tenant account to the tenancy record.

Ownership checks are a second and separate layer. Every read is scoped by owner, but a write that accepts an identifier from the request body needs an explicit check that the row it names belongs to the caller. Those checks now live in one helper module rather than being written out at each site. There is a reason for that, recorded in Chapter 6: when they were written out individually, one of three endpoints was missed.

Schema changes apply themselves at boot, and apply safely more than once. The schema file runs its table creation statements conditionally on every start, and column-level migrations run behind existence guards. No deploy has needed a manual migration step.

4.7 Payment and Settlement Design

There are two payment routes and both are direct.

The first is UPI. The tenant's screen carries the landlord's own virtual payment address, the exact amount and a generated reference, encoded as a scannable code and as an intent that hands off to whichever UPI application the tenant already uses. The money goes from the tenant's bank to the landlord's. Nobody pays a fee, including the platform.

The second is a payment gateway. A landlord can connect their own Razorpay merchant account by entering their key and secret, which are checked against the gateway before being stored and are then sealed with authenticated encryption and never returned to any client. When a tenant pays this way the server creates a payment link against that landlord's own credentials; the tenant pays on the gateway's hosted page and the money settles to that landlord. The gateway charges its fee to the landlord through the landlord's own account. The platform bills nothing, because it is party to nothing.

Payment links were chosen over the gateway's React Native software development kit for the reason given in section 4.5. The kit is a native module. A link is plain REST over HTTPS, so the payment feature ships over the air like every other change.

The confirmation model ties the two routes together. A payment enters as a declared row, which is a claim and clears nothing. It becomes real when a second party vouches for it: the landlord tapping Confirm, or the gateway being asked outright, server to server, with that landlord's credentials, and answering that the link was paid. Both paths end in the same settlement function, which flips the status and moves the tenancy's next due date forward inside one transaction. Write that twice and the two copies will eventually disagree about when a month has cleared, so it is written once.

The gateway gets polled rather than trusted to call back. A webhook would mean every landlord configuring a URL and a shared secret in a dashboard, and most of them would never finish doing it. The signed webhook path exists and is still supported; it is just the secondary route.

4.8 Session and Authentication Design

The session design went through three versions. The sequence is worth recording, because each of the first two fixed something real and created the next problem while doing it.

Table 4.1 — Evolution of the Session Design

Stage	Access token	Can be revoked	Failure it solved	Failure it left
Original	7 days, fixed	No	—	Weekly logout for every user
Sliding	30 days, renewed on use	No	The weekly logout	Nothing could be cancelled
Current	2 hours plus a refresh token	Yes	Revocation	Lag of up to two hours on the access path

Raising the lifetime from seven days to thirty did not stop people being logged out. It made it happen monthly. A fixed lifetime counts from the last sign-in, which says nothing about whether somebody is still using the application. Open the app every morning and you were thrown out on day thirty, exactly like somebody who had not opened it since February. So the server now re-issues the access token on any authenticated request once it is past half its life, handing it back on a response header the client picks up. The new

token's lifetime comes from the old one's own issue and expiry claims rather than from a constant, which is what lets a guest session keep its own length.

A session is now a database row as well as a signature. The row holds a refresh token stored as a SHA-256 hash rather than as the token itself, since the only thing ever done with it is a comparison. Three properties follow. Rotation: spending a refresh token gets you a replacement, so a sixty-day secret is never sent repeatedly over the network. Reuse detection: the honest client always holds the newest token, so a replay of a spent one says a copy is sitting somewhere else, and every session on that account is revoked. Revocation on password reset: changing a password now ends every other session, which is what users had assumed all along that it did.

Revocation is immediate on the refresh path. On the access path it lags by at most the access token's two-hour life. Closing that last gap would mean a database read on every authenticated request, which is a cost almost nobody pays. Two hours where it used to be thirty days is the gain.

4.9 Database Design

The schema holds 22 tables and follows the entity structure closely. Landlord accounts and tenant accounts are separate tables, which is what lets identity and authorisation be told apart at the API layer. Properties belong to landlords, rooms belong to properties, and tenancies join a tenant record to a room with their own rent, deposit, split and dates. Payments reference the tenancy and carry the due date they settled, and that last field matters more than it looks. Without it, "was this on time?" has no answer after the fact, because confirming a payment moves the next due date forward and overwrites the only evidence. Its earlier absence is why the behaviour score once reported that no tenant had ever missed a payment.

Table 4.2 — Principal Tables and Their Role

Table group	Holds	Notes
Accounts	Landlord accounts, tenant accounts	Separate tables with independent keys
Portfolio	Properties, rooms	Rooms carry rent, capacity and notification preferences
Tenancy	Tenant records, assignments, notice periods	Due dates derived from each tenancy's own start date
Payments	Declared, confirmed and rejected payments	Each row stores the due date it settled

Table group	Holds	Notes
Maintenance	Requests and thread messages	Status transitions stored as events in the thread
Documents	Identity proofs and maintenance photographs	Stored externally; previews served under short-lived tokens
Notifications	Alerts, delivery records, seen markers	Seen marker is stamped by the server, not the device
Sessions	Refresh token hashes, expiry, revocation reason	Hash only; the token itself is never stored
Gateway	Per-landlord sealed gateway credentials	Encrypted with AES-256-GCM; never returned to a client

The entity relationship diagram below shows the core of the schema with its keys and cardinalities. The chain from an owner down to a payment runs through four tables, which is why an ownership check on a write that names a room is not local to the row being written and had to be factored into a shared helper.

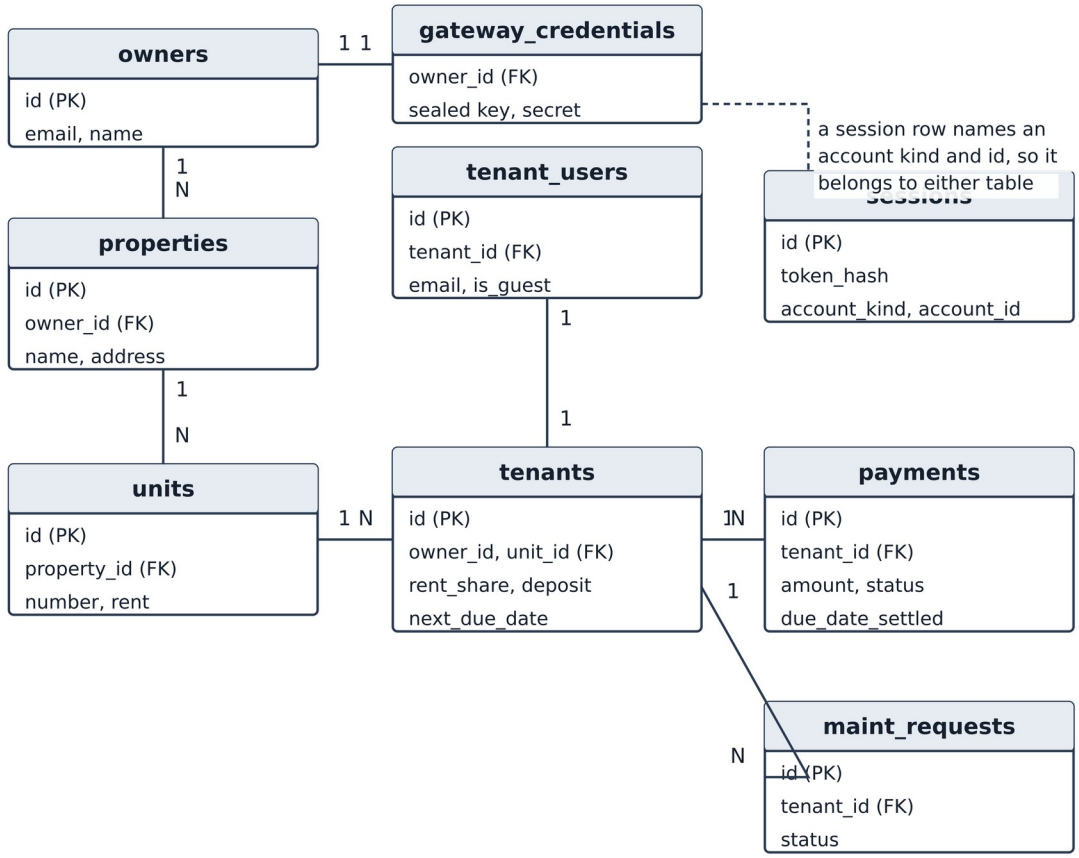


Fig 4.6 — Entity relationship diagram of the core schema

CHAPTER 5 — SYSTEM REQUIREMENTS

5.1 Hardware Requirements

TenantPro is a client-server system. On the user's side the requirements are small: an Android phone of reasonably recent vintage and an internet connection. On the server side they are those of a small Node process and a MySQL instance, both of which run comfortably on entry-level cloud infrastructure at the scale this system is built for.

Table 5.1 — Hardware Requirements

Component	Minimum	Recommended
End-user device	Android 8.0 smartphone, 2 GB RAM	Android 11 or later, 4 GB RAM
Network	2 Mbps mobile data	4G or Wi-Fi
Application server	1 vCPU, 512 MB RAM	2 vCPU, 2 GB RAM
Database server	1 vCPU, 1 GB RAM, 10 GB storage	2 vCPU, 2 GB RAM, 25 GB storage
Development machine	4 GB RAM, 20 GB free disk	8 GB RAM or more, SSD

5.2 Software Requirements

The stack was picked to balance productivity, delivery flexibility and cost.

Table 5.2 — Software Requirements

Layer	Technology	Purpose
Client framework	React Native on Expo SDK 54	Single codebase for both roles, Android build
Client delivery	Expo updates and EAS Build	Over-the-air JavaScript updates; store builds when native changes
Server runtime	Node.js with Express	REST API, 87 endpoints across role-guarded routers
Database	MySQL	22 tables; schema applied idempotently at boot
Authentication	Signed access tokens with server-side refresh tokens	Short-lived access, revocable sessions
Encryption	AES-256-GCM, SHA-256	Sealed gateway credentials; hashed refresh tokens
Payments	Razorpay Payment Links over REST; UPI intent and QR	Direct settlement to the landlord
Messaging	Transactional email, push notifications, SMS and WhatsApp	Rent reminders and alerts
Media	Cloud media storage	Identity documents and maintenance photographs

Layer	Technology	Purpose
Testing	Plain Node scripts; Playwright with Chromium	Unit, integration, end-to-end and browser-driven suites
Version control	Git and GitHub, with GitHub Actions	Source control and build or update workflows

5.3 Programming Platform Features

React Native with Expo gives a single JavaScript codebase that runs on the device while still letting most changes go out without a store release. That second property is not a convenience. It is a constraint, and it shaped the feature set in the way described in Chapter 4.

Node with Express was picked for the server because JavaScript on both sides removes a translation layer between client and server models, and because the Express router model maps neatly onto the role-guard design. MySQL was picked over a document store because this data is strongly relational: a payment means nothing except in relation to a tenancy, which means nothing except in relation to a room and a property.

External integrations are reached through documented REST APIs with explicit error handling. One point of care is worth flagging here and comes up again in Chapter 6. A request that never reached a third party is a different event from one that reached it and was refused, and the two must not be reported to the user in the same words.

5.4 Functional Requirements

The system shall let a landlord register, sign in and recover a forgotten password. It shall let them create properties and rooms, with per-room rent, capacity, occupancy status and notification preferences. Tenants shall be creatable directly or able to join by redeeming an invite code, with the landlord accepting or declining and optionally assigning a room at the same time.

It shall work out each tenancy's due date from its own start date and maintain every tenant's due state. It shall let a tenant declare a payment, and shall record that payment as a claim carrying the due date it is meant to settle. It shall let a landlord confirm or reject a claim, with a rejection reason the tenant can read. Where a landlord has connected a gateway, it shall create a payment link against that landlord's credentials and shall confirm the payment by asking the gateway rather than by trusting the client.

It shall keep a searchable ledger of confirmed payments, narrowable to one property. It shall send rent reminders daily on the configured channels. It shall provide maintenance threads shared between landlord and tenant, with status transitions recorded in sequence. It shall accept identity documents and serve previews under short-lived tokens. It shall compute a payment-behaviour score from the same code for both parties and itemise its factors. It shall record notice periods and compute leave dates on the server. And it shall give the tenant a portal showing their dues, history, receipts, deposit, tenure and score.

5.5 Non-Functional Requirements

Usability comes first, because the users are not necessarily technical. Screens are expected to state the cost and the prerequisites of an action before the user commits to it, and error messages are expected to tell a network failure, a refusal by a third party and a mistake by the user apart from one another.

Security requires third-party credentials to be stored encrypted and never returned to a client, refresh tokens to be stored only as hashes, sessions to be revocable, and authorisation to be enforced on the server for every write that accepts an identifier from the client. Correctness requires that a payment is never treated as settled on the payer's assertion alone, and that a status change and a due-date advance happen inside one transaction.

Reliability requires that a passing network failure never signs a user out and is never reported as a rejected credential. Maintainability requires that behaviour shared between the two applications is implemented once. Portability requires that schema changes apply themselves at boot, so that a deploy is a code deploy and nothing more.

CHAPTER 6 — IMPLEMENTATION

Implementation went module by module, each module being a functional area with its own routes on the server and its own derived state on the client. They are described below roughly in the order a landlord and a tenant run into them.

6.1 Authentication and Session Module

This module registers accounts, authenticates them, issues sessions and ends them. Landlord and tenant accounts sit in separate tables, and the token issued to each carries a role the API's guards act on. Sign-in can also go through a federated Google identity.

A session is a short-lived access token plus a refresh token held as a hash in a sessions table. The access token is renewed quietly on any authenticated request once it passes half its life. The refresh token rotates every time it is used, and presenting one that has already been spent revokes every session on the account. Signing out revokes the row with a recorded reason, and a password reset revokes every other session.

The client half needed the most care. A failed request triggers at most one refresh and one replay, and only a refusal from the refresh endpoint itself ends the session; a server fault, a timeout or a dead connection does not. Parallel refreshes all share one in-flight promise, so an application firing eight requests on a cold start issues a single refresh instead of eight. Without that, seven of the eight would replay a token the first had already spent, the server would correctly read the replay as theft, and the feature built to stop lockouts would have been causing them.

6.2 Property, Room and Tenant Management Module

This is where the landlord describes the shape of the business. Properties get a name, a type and an address. Rooms get a number, rent, capacity and their own notification preferences. A room's occupancy status follows its occupants on its own.

Tenants can be created directly by the landlord or can join by redeeming an invite code, in which case the landlord sees the request with the tenant's details and any identity documents before accepting it. A tenant is allowed to exist without a room, and that state is reachable from the roster rather than tucked out of sight, because somebody added but not yet placed is an ordinary situation and not an error.

Every write in this module that takes a room identifier from the request body checks that the room belongs to the caller. That check sits in one shared helper. It did not always, and section 6.10 says what came of it.

6.3 Rent Cycle and Due Date Module

Each tenancy carries its own start date, rent and split. The due date is computed from the tenancy's own anniversary rather than from a shared calendar month, which is what lets a tenant who moved in on the nineteenth be due on the nineteenth. The module keeps every tenant's due state and exposes it as the chip shown on the roster: due in so many days, so many days late, or a leaving date where notice has been given.

Advancing the due date is not done here. It happens only inside the settlement transaction described in section 6.4, so a due date can never move without a payment having settled, and a payment can never settle without the due date moving.

6.4 Payment and Confirmation Module

A payment is created as a declared row carrying the amount, a generated reference, the method, and the due date it is meant to settle. In that state it is a claim and clears nothing.

Confirmation comes from one of two places. The landlord can confirm or reject the claim from a queue showing the context needed to decide, and a rejection carries a reason the tenant reads. Or, where the payment went through a connected gateway, the server asks the gateway directly using that landlord's credentials and settles on the gateway's own answer. Both call the same settlement function, which flips the status and moves the tenancy's next due date forward in one transaction.

On the tenant's side the distinction is made visible rather than hidden. A claim still waiting on the landlord shows in amber as awaiting confirmation, not in the green kept for money that has arrived. Show the two alike and a tenant will believe their month is clear when it is not.

6.5 Payment Gateway Connection Module

This module lets a landlord connect their own Razorpay merchant account. The sheet that does it states the gateway's fee right next to the offer, because describing one without the other is how somebody switches on a percentage fee without noticing. It also sets out what the gateway is going to ask for before the landlord starts: a permanent account

number, a bank account, one business proof, and realistically two to five working days. It adds that test keys work straight away and that tenants can carry on paying by UPI the whole time.

Credentials are checked against the gateway before anything is stored, so a mistyped secret is caught while the landlord still has the form in front of them. What gets stored is sealed with AES-256-GCM and never returned to any client. A failure to reach the gateway is reported as a network failure and not as a rejected credential; the word "rejected" is kept for the gateway actually saying no.

6.6 Notification and Reminder Module

A scheduled job runs each morning and sends rent reminders to the tenants who are actually due, across email, push, SMS and WhatsApp according to the room's preferences, with every channel phrasing the same day the same way. Beyond reminders, the module raises alerts for a landlord's reply on a maintenance thread, a payment confirmed or rejected, and the outcome of a join request.

The alert indicator works off the same payload the screen renders rather than a separate unread counter that could drift away from it. A seen marker tells an alert that is news apart from one that is merely still true, and the server stamps that marker, so a phone with the wrong clock cannot silence its own alerts.

6.7 Maintenance Request Module

A tenant raises a request with a category, a priority and a photograph if they have one, then follows it in a thread shared with the landlord. Status changes land in that thread as events, in order with whatever was said around them, so the history of a repair reads as one story instead of a conversation plus a separate status field.

6.8 Tenant Behaviour Score Module

The module computes a payment-behaviour score from the tenant's record of on-time, late and missed payments. One piece of code produces it and both sides see the same number, which is the whole point: a tenant looking at a different figure from the one their landlord sees cannot argue with either. The factors are listed out instead of being boiled down to a single dial, and where there is too little history to say anything, the screen says so. The alternative is a needle parked at zero, which reads like a verdict the tenant has earned.

6.9 Tenant Portal Module

The tenant portal is a parallel application rather than a cut-down copy of the landlord's. Its home screen carries the rent card with the amount, the property and room, the exact due date and a countdown, and underneath it the deposit held and the tenure. Both of those are things tenants routinely cannot remember and have no other record of.

The module also covers guest accounts for tenants with no email address, identified by a code with a stay window enforced on the server; turning a guest into a full account; joining a property by scanning or typing an invite code; receipts with status, reference and any rejection reason; giving and withdrawing notice, with the real dates worked out and shown before anything is written; uploading and removing identity documents; and directions to the property where the landlord has pinned it.

6.10 Testing

The build carries 457 assertions across eight suites. None of them uses a test framework. Each suite is a plain Node script that exits non-zero when something fails, which keeps the test code free of dependencies and makes a failing run reproducible from one command.

Table 6.1 — Test Suites and Assertion Counts

Suite	What it drives	Isolation	Assertions
secretbox	AES-256-GCM sealing of gateway credentials	Pure	24
session	Token renewal, through the real middleware and interceptor	Stubbed database	41
gateway	Gateway controllers	Stubbed database and network	72
vm.gateway	The real view-model function, native layer stubbed	Stubbed native	52
refresh.client	The real request interceptor: refresh, retry, and when not to sign out	Stub adapter	29
refresh.e2e	Refresh tokens, reading the database directly	Live server and MySQL	42
api.e2e	All 87 endpoints	Live server and MySQL	126
ui.e2e	The compiled application in a browser engine	Live stack and Chromium	71
Total	—	—	457

6.10.1 Unit Testing

Unit tests cover the calculation-heavy and security-critical functions on their own: sealing and unsealing gateway credentials, hashing and comparing refresh tokens, the renewal arithmetic that reads a token's remaining lifetime out of its own claims, and the view-model function that turns server state into what the screens render.

6.10.2 Integration Testing

Integration tests run the controllers with their dependencies stubbed at the boundary rather than inside. The gateway suite is the clearest example: it runs the real controller code against a stubbed network, which is how the difference between a refused credential and an unreachable host can be asserted directly instead of inferred.

6.10.3 End-to-End Testing

The end-to-end suites run against a live server and a real MySQL instance seeded by a deterministic generator. That generator deliberately produces the rows that break screens and never turn up when you want them: a tenant with no room, a guest with no email, a guest whose stay has already ended, rent overdue and due today and upcoming and never set at all, a payment declared and undecided, another rejected with a reason, maintenance threads where the landlord spoke last and others where the tenant did, a tenant who has given notice, and a property with no rooms in it. The pseudo-random number generator is seeded, so a failing run can be reproduced exactly.

The refresh suite goes further and reads the sessions table directly, so claims about rotation, reuse detection and revocation are checked against stored rows rather than against the API's account of itself. During one live browser run a session was driven through three access-token expiries without the user being signed out. Signing out through the interface afterwards was confirmed to have written a revocation reason into the row and to have made the token unusable.

6.10.4 Browser-Driven User Interface Testing

The user interface suite runs the same JavaScript the phone runs: the same view-model, the same screens, the same request client and interceptors, laid out for the browser by the web renderer an Expo web export produces, and driven through Chromium. What it cannot reach is anything behind a native module, so the camera, the image picker, push

delivery and the UPI intent hand-off are all out of its scope. For those it prints a skip rather than a pass, so a green run never claims coverage it does not have.

Running the application in a desktop browser is not a substitute for a device and is not claimed as one here. It is, though, the only automated way available in this project's environment to assert that a real screen renders a real row correctly, and it found defects no unit test had.

6.10.5 Defects Found

Driving every endpoint against a real database, and the application against a real browser, turned up five defects the unit tests had missed.

Table 6.2 — Defects Found by the End-to-End Runs

Defect	Severity	Status
Two of three endpoints accepting a room identifier checked ownership; the third did not, allowing one landlord's tenant to be filed into another landlord's room	Cross-account write	Fixed; the check is now one shared helper
A wrong password rendered "Your session expired", overriding the server's own message	User-facing	Fixed
Five required-field omissions produced HTTP 500 with a raw database error instead of a 400	Robustness	Fixed
Renaming a property without its address crashed on a NOT NULL column coerced on insert but not on update	Robustness	Fixed
A network failure reaching the gateway was reported as a rejected credential	User-facing	Fixed; transport failures now distinguished

The first one is worth dwelling on, since it is why ownership checks were centralised. Every read in the system is scoped by owner, so the second landlord never saw the intruding row. The leak was that the first landlord's own tenant list then displayed a stranger's property name and room number. We found it by filing one landlord's tenant into another's room through the API and getting a cheerful success response back.

Two more defects turned up in the test harnesses themselves, which is worth putting on the record. The first browser run never got past the permissions prompt, and because its assertions were phrased as the absence of a string that the prompt also happened to lack, half the suite passed while testing nothing at all. Separately, the failure reporter crashed on a failure whose context was undefined and took down the run it was meant to be reporting. The lessons are to assert on what should be there rather than on what should not, and to keep the failure reporter the dullest code in the file.

CHAPTER 7 — RESULTS AND SCREENSHOTS

The finished system meets the requirements set out in Chapter 5. A landlord can run several properties and their rooms, track every tenant's dues, confirm or reject payments, handle maintenance threads and connect a payment gateway. A tenant can see what they owe, pay it by either route, follow their own history, raise maintenance requests and give notice. The screenshots in this chapter come from the running application working against a seeded database of three properties, six rooms and ten tenants. None of the figures in them are placeholder text.

7.1 Landlord Overview

The landing screen answers the two questions that make a landlord open the application at all: who has paid, and who has not. Collection is shown against what was expected rather than as a bare total, because a number with no denominator is not something anyone can act on. Overdue gets a card of its own with a count of people attached. The revenue series is built from confirmed payments only. Claims that have been declared but not yet confirmed are left out on purpose, since counting them would inflate income on nothing more than a tenant's word. The bell carries a live count of whatever is waiting on a decision.

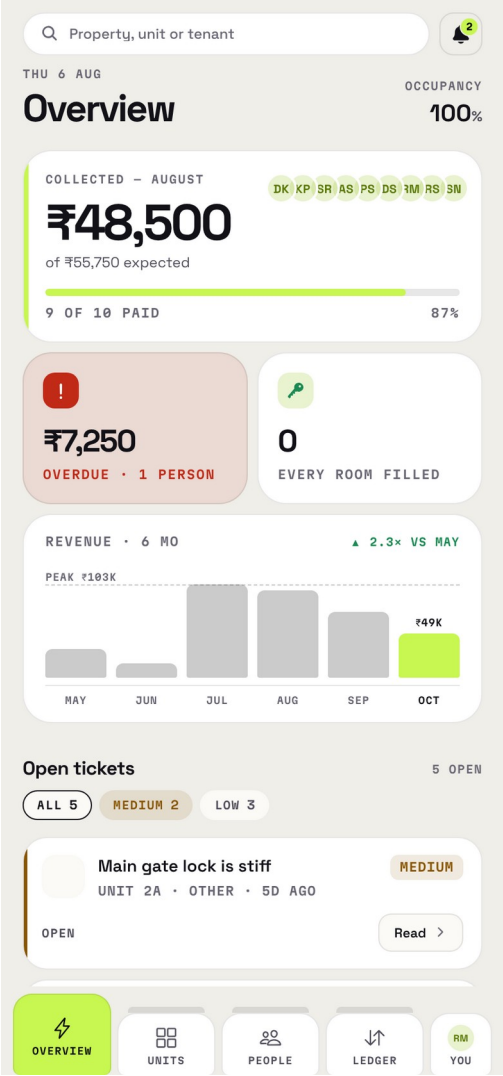


Fig 7.1 — Landlord overview: collection against expectation, overdue total, occupancy, six-month revenue and the open-ticket queue

7.2 Tenant Roster

Each row carries the tenant's property and room, their share of the rent, a payment-behaviour score and a due-state chip. The filter chips carry counts, so the spread is visible before any of them is tapped. The unassigned filter is not an afterthought. A tenant is allowed to exist without a room, and the end-to-end suite checks that such a row renders without a broken room label.

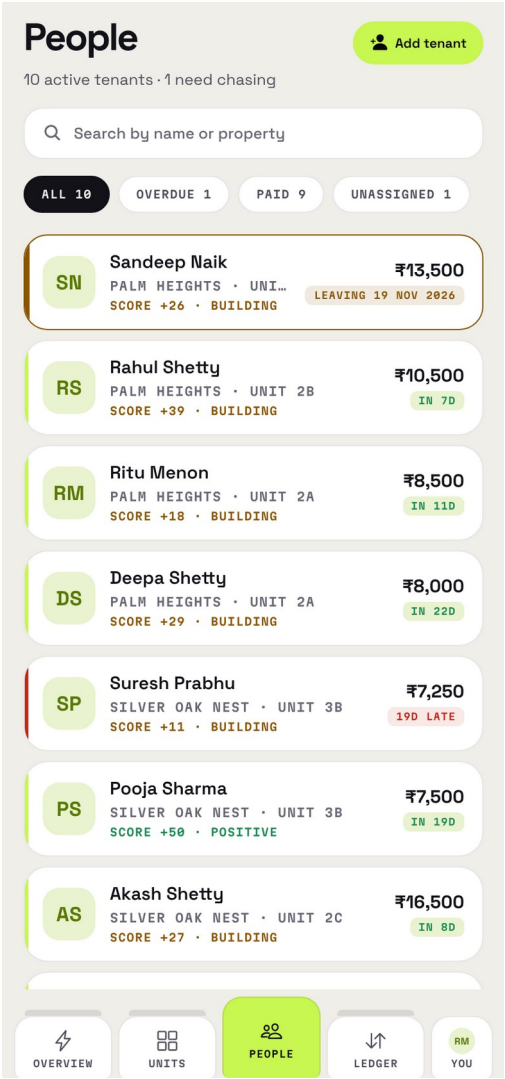


Fig 7.2 — Tenant roster with counted filters, due-state chips and behaviour scores

7.3 Rooms and Properties

Properties hold rooms and rooms hold tenants, each room with its own rent, capacity, and a split across occupants that can be equal or set by hand. A room's status follows its occupancy on its own. Notification preferences are per room, with each channel switched independently, because a landlord rarely wants the same channel across a whole portfolio.

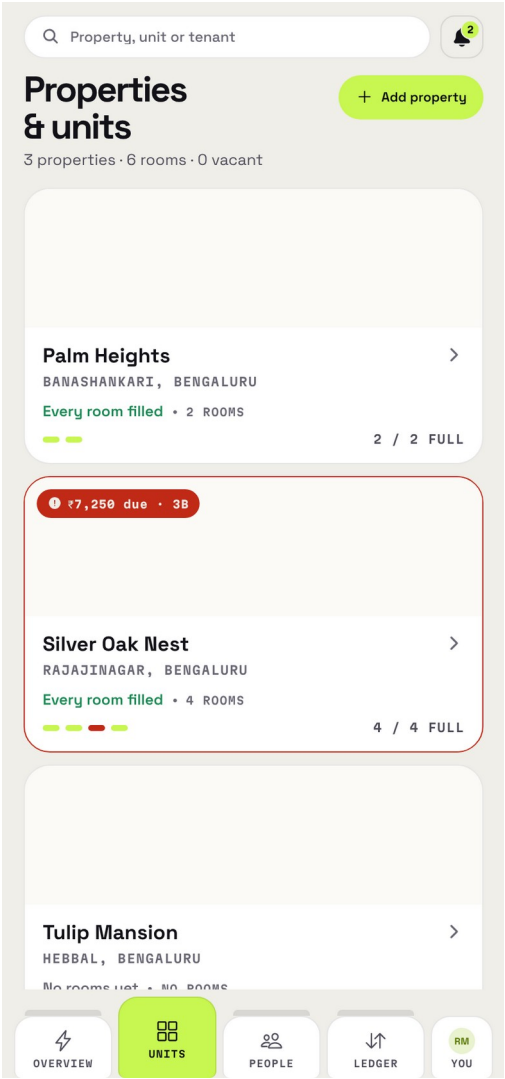


Fig 7.3 — Room list showing occupancy, rent and tenant assignment per unit

7.4 Ledger

The ledger holds every confirmed payment, searchable and narrowable to a single property. Each payment stores the due date it settled, captured at the moment of declaration, and that is the field that makes punctuality answerable after the fact.

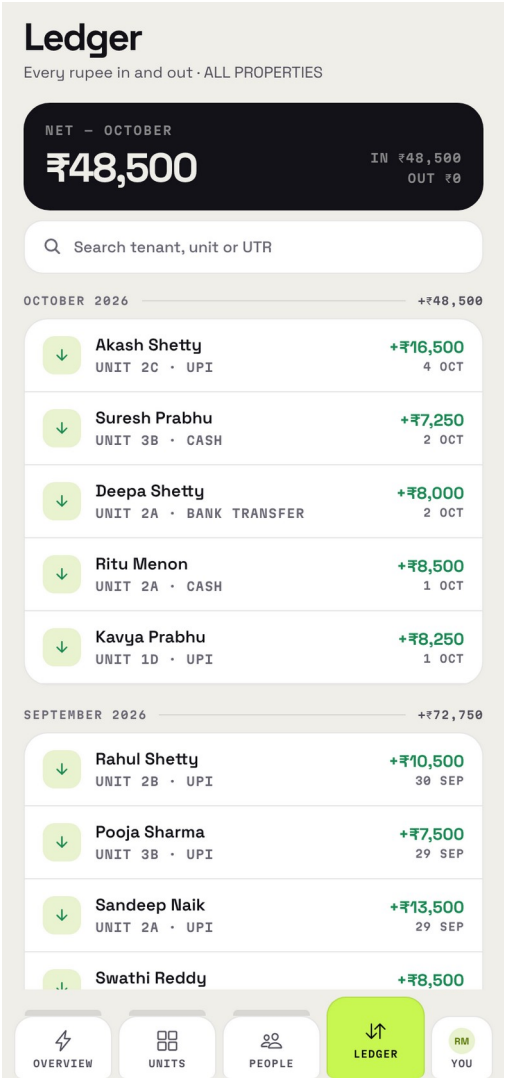


Fig 7.4 — Ledger of recorded payments with method, date and tenant

7.5 Connecting a Payment Gateway

The sheet states the fee next to the offer, and sets out what the gateway will ask for and how long it will take before the landlord begins. Credentials are checked against the gateway before being stored and sealed afterwards.

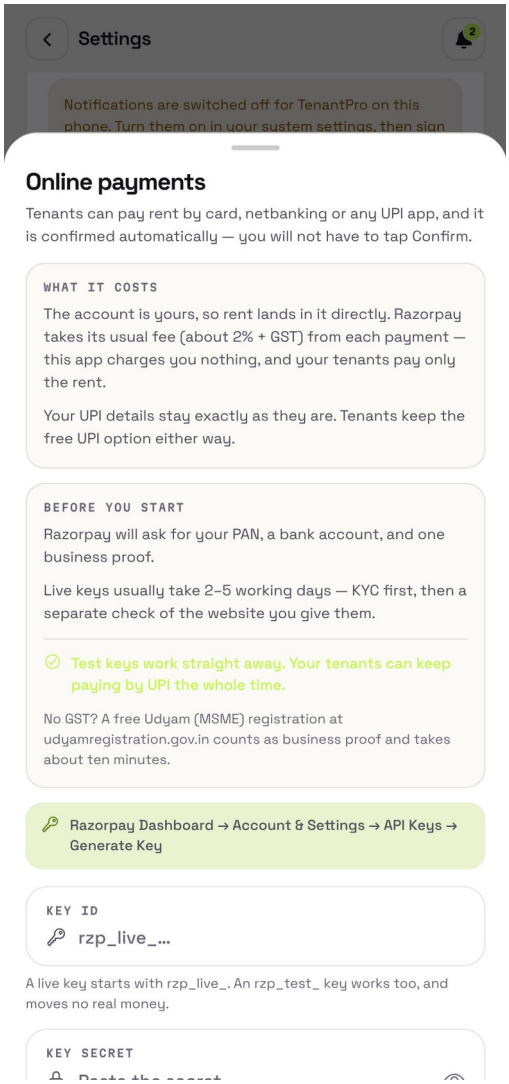


Fig 7.5 — The online payments sheet: what it costs, what the gateway will ask for, and the key fields

7.6 Tenant Home

The rent card carries the amount, the property and room, the exact due date and a countdown to it. Deposit held and tenure sit underneath, with the landlord's contact and any open requests after them. The alert indicator works off the same payload the screen renders rather than a separate counter.

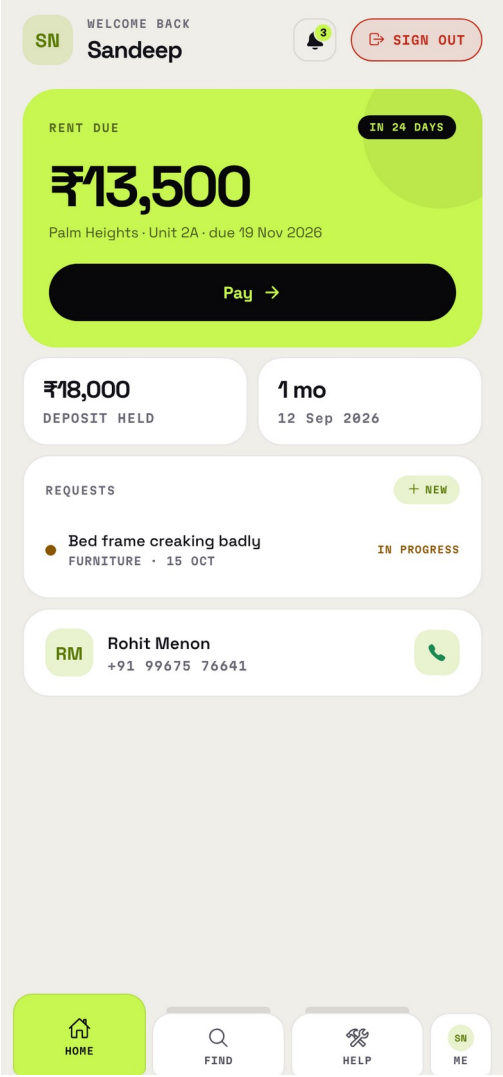


Fig 7.6 — Tenant home: rent due with countdown, deposit held, tenure, open requests and the landlord's contact

7.7 Paying Rent

The screen shows what is owed, a reference, and zero as the platform fee, which is literally true on both routes. Where the landlord has connected a gateway, the card and netbanking route comes first, since it is the only one that confirms itself. Below a divider sits the UPI route, with the landlord's own address, the amount and the reference encoded for scanning from another phone and as a hand-off to a UPI application on this one. The two buttons are styled differently on purpose: two filled primary buttons amount to two primary actions, which is the same as having none. Coming back from a gateway page, the button reads "Check my payment" rather than "Yes, I paid", because on that route the answer is not the tenant's to give.

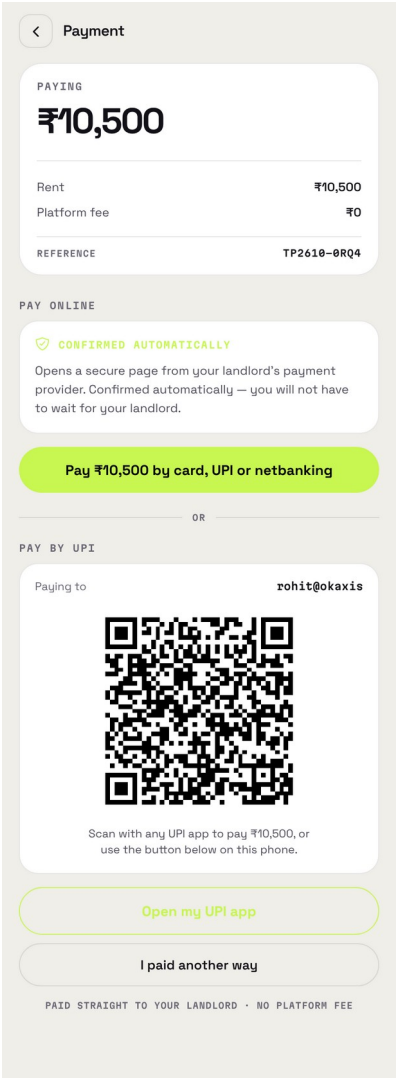


Fig 7.7 — The payment screen with a gateway connected: the online route above the divider and the UPI route beneath it

7.8 Tenant Profile and Payment History

The profile pulls together tenure, rent share, deposit and the behaviour score with its factors listed out, followed by the payment timeline. The timeline is built from real rows, and a claim still waiting on the landlord shows amber rather than the green kept for money that has arrived.

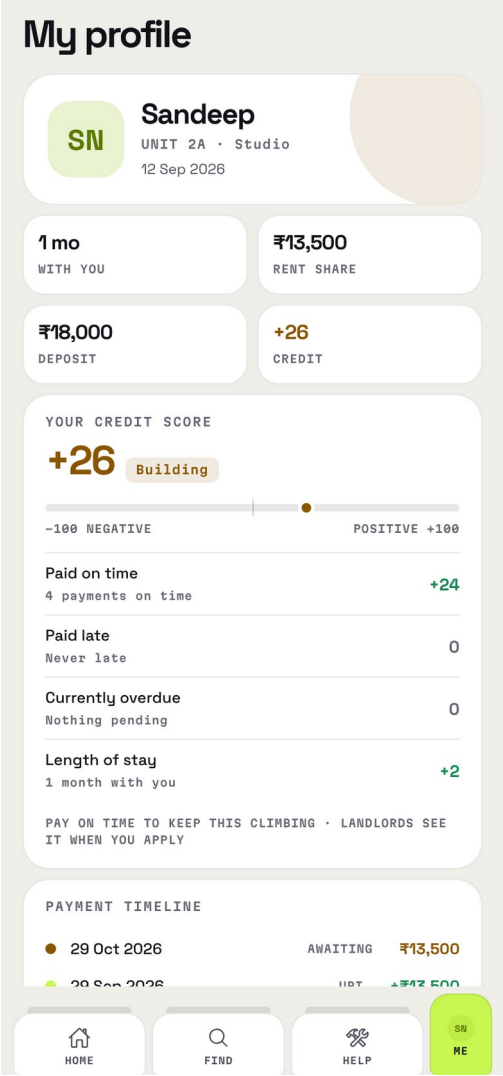


Fig 7.8 — Tenant profile: tenure, rent share, deposit, the behaviour score with its factors, and the payment timeline

7.9 Maintenance

A tenant raises a request with a category, a priority and an optional photograph, and follows it in a thread shared with the landlord, with status changes appearing in sequence alongside the conversation.

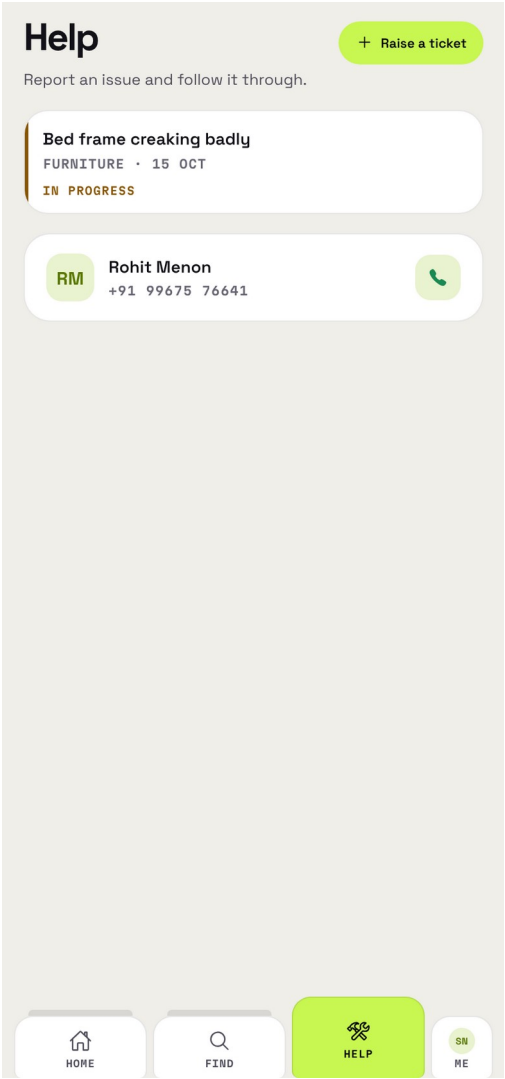


Fig 7.9 — Tenant help screen listing maintenance requests with their status

CHAPTER 8 — CONCLUSION AND FUTURE ENHANCEMENT

8.1 Conclusion

This project set out to deal with a specific and long-standing gap in the independent rental sector: manual record keeping still dominates, and because of it there is no shared, authoritative account of what a tenant owes and what they have paid. What we built to address it is a working two-sided application, deployed and in use, made of a React Native client and a Node and Express API over MySQL, running to 38,536 lines across 124 source files, 87 endpoints, 22 tables and 28 screens.

What separates it from the alternatives reviewed in Chapter 2 is that the platform never touches the money. Rent goes straight from the tenant to the landlord, by UPI or through the landlord's own merchant account, and the platform only states the correct amount and destination and records what happened. That keeps the operator outside the payment-aggregator perimeter, costs both parties nothing, and leaves settlement where the landlords we spoke to wanted it. The price is that the system cannot observe money it does not handle, and the confirmation model exists to pay that price honestly rather than to pretend it does not apply.

The verification work produced the most useful findings of the project. Exercising all 87 endpoints against a real database and the compiled application against a real browser engine turned up five defects that unit tests over mocks had not, among them a cross-account authorisation hole that let one landlord's tenant be written into another landlord's room. Two further defects were in the test harnesses. The experience supports a conclusion that is well established in the literature and still easy to ignore in a student project: tests that assert on the absence of a string, and systems only ever tested against their own mocks, give false confidence.

More broadly, the project shows that an independent landlord does not have to choose between a paper register and handing custody of their rent to a platform. A system can give both sides a complete, shared record while leaving the money exactly where it already flows.

8.2 Limitations of the Present Work

These are set out plainly, since each one follows from a design decision rather than from an oversight. The application cannot observe money. It never sees a bank balance, so a UPI payment stays a claim until the landlord confirms it, and for a landlord who has not connected a gateway the confirmation step is structural rather than incidental. Revocation lags by up to two hours on the access path, because closing that gap completely would mean a database lookup on every authenticated request.

Reuse detection can produce a false positive. A client that stores a new access token but fails to store the refresh token issued with it will present a spent token and lose every session. The client-side write is transactional, but a device that loses its storage halfway through would still land here. SMS is unconfigured, because phone verification needs a telecom registration that has not been done; email and push are both live. No iOS build has ever been produced, the bundle identifier is unset, and the project has only ever turned out Android artifacts.

Gateway onboarding is heavy. A landlord connecting Razorpay has to get through that gateway's own verification, realistically two to five working days, and although the application now says so before they start, it is still the biggest piece of friction in the product. Finally there is no automated regression gate. The suites exist and they pass, but nothing runs them on push, so verification stays a manual step somebody has to decide to take.

8.3 Future Enhancement

Several directions have been identified. Some are small and specified; others are larger and need more thought.

The immediate items are well understood. Reply notifications on maintenance threads are incomplete: a tenant replying to a repair request currently notifies nobody, though the landlord-to-tenant direction already works. The landlord's own payment-entry path inserts the payment and updates the due date in two separate statements where the tenant-facing path is already transactional, and that asymmetry should go. The suites are reproducible and fast enough to run on every push, so wiring them into continuous integration is configuration work rather than development work. An iOS build needs a bundle identifier and a first run through the pipeline. And a landlord announcement feature, a one-to-many

message to everyone in a property, was specified during the notification work and then never built, although the alert infrastructure that would carry it already exists.

The larger directions need more care. Split settlement through a gateway's linked-account facility would let landlords onboard from inside the application, with the gateway doing their verification, which would remove the onboarding friction named above. The cost is that funds would then pass through a platform account, and that is a change of regulatory category rather than of configuration. The 2025 directions restrict such arrangements in ways that call for professional advice rather than inference. Worth revisiting when the user base justifies it. Not before.

Monetisation, if it is ever pursued, should stay out of the money path. The Namma Yatri precedent applies here almost directly: a flat per-landlord subscription keeps the operator outside the payment flow, whereas a percentage of each transaction starts to look structurally like aggregation. There is a related legal nuance, which is that models of this kind rest on selling access to software rather than on setting the price of the underlying service. In TenantPro the landlord sets the rent, and that is a position worth holding on to deliberately.

Two smaller pieces of work are already identified on the messaging side. Transactional mail currently goes out from a consumer address through a provider's HTTPS interface; a verified sending domain with the usual authentication records would improve inbox placement a good deal, for about a thousand rupees a year. And phone verification needs an Indian telecom registration before any operator will carry template traffic.

REFERENCES

[1] Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.

[2] Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.

[3] Elmasri, R., & Navathe, S. B. (2017). Fundamentals of Database Systems (7th ed.). Pearson.

[4] Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill.

[5] Fowler, M. (2018). Refactoring: Improving the Design of Existing Code (2nd ed.). Addison-Wesley Professional.

[6] Nielsen, J. (1993). Usability Engineering. Morgan Kaufmann Publishers.

[7] Reserve Bank of India. Guidelines on Regulation of Payment Aggregators and Payment Gateways, and subsequent amendments. Retrieved from https://www.rbi.org.in/

[8] Commentary on the Regulation of Payment Aggregators Directions, 2025 — industry analysis of the revised framework for payment aggregators.

[9] Razorpay Developer Documentation. Payment Links API. Retrieved from https://razorpay.com/docs/payments/payment-links/

[10] Razorpay Developer Documentation. API Keys, and the website verification required for live mode. Retrieved from https://razorpay.com/docs/

[11] Razorpay. Route: split settlement and linked accounts. Retrieved from https://razorpay.com/docs/payments/route/

[12] Razorpay. KYC documents required by business type. Retrieved from https://razorpay.com/docs/

[13] National Payments Corporation of India. UPI Linking Specification and Deep Linking Guidelines. Retrieved from https://www.npci.org.in/

[14] Namma Yatri. Zero-commission direct-payment model — public documentation of the open mobility initiative. Retrieved from https://nammayatri.in/

[15] Internet Engineering Task Force. RFC 7519: JSON Web Token (JWT). Retrieved from https://www.rfc-editor.org/rfc/rfc7519

[16] Internet Engineering Task Force. OAuth 2.0 Security Best Current Practice, including refresh token rotation and replay detection. Retrieved from https://datatracker.ietf.org/doc/html/draft-ietf-oauth-security-topics

[17] National Institute of Standards and Technology. SP 800-38D: Recommendation for Block Cipher Modes of Operation — Galois/Counter Mode (GCM). Retrieved from https://csrc.nist.gov/

[18] Expo Documentation. Expo Updates and EAS Build. Retrieved from https://docs.expo.dev/

[19] React Native Documentation. Retrieved from https://reactnative.dev/

[20] Node.js Foundation. Node.js Official Documentation. Retrieved from https://nodejs.org/en/docs/

[21] Express.js. Express Web Framework Documentation. Retrieved from https://expressjs.com/

[22] Oracle. MySQL 8.0 Reference Manual. Retrieved from https://dev.mysql.com/doc/

[23] Microsoft. Playwright Documentation — browser automation for end-to-end testing. Retrieved from https://playwright.dev/

Note on secondary sources. Several of the regulatory points in this report rest on consultancy and industry commentary rather than on the Reserve Bank of India's own text. Where the report leans on them, particularly for the net-worth thresholds and the restrictions on split settlement, the claims should be checked against the Directions themselves before anybody makes a structuring decision. The code figures, endpoint counts and assertion totals were measured directly from the repository and from live test runs, and are not estimates.